

03 - Problema Wake-up

In un sistema distribuito costituito dall'insieme delle entità V , il problema del **wake-up** consiste nel raggiungere una configurazione finale del sistema in cui tutti i nodi sono nello stato **Awake**, a partire da una situazione in cui un sottoinsieme di nodi $W \subseteq V$ iniziatori si trovano nello stato **Awake** e i restanti nodi in $V \setminus W$ si trovano nello stato **Asleep**.

Il problema

Formalmente, il problema è descritto dalla tripla $P = \langle P_{init}, P_{final}, R \rangle$ in cui, sia $W \subseteq V$ tale che $W \neq \emptyset$:

- $P_{init} = \forall x \in V, s(x) = \begin{cases} \text{awake} & x \in W \\ \text{asleep} & x \in V \setminus W \end{cases}$
- $P_{final} = \forall x \in V, s(x) = \text{awake}$
- $R = \begin{cases} \text{Total Reliability (TR)} \\ \text{Bidirectional Link (BL)} \\ \text{Connectivity (CN)} \end{cases}$

Protocollo WFLOOD

Per risolvere il task del Wake-Up, si progetta il seguente protocollo, che prende il nome di **WFLOOD**.

Durante l'esecuzione del protocollo, le entità assumono i seguenti stati:

- $S = \{\text{awake}, \text{asleep}\}$
- $S_{init} = \{\text{awake}, \text{asleep}\}$
- $S_{term} = \{\text{awake}\}$

il protocollo è descritto dal seguente pseudocodice

```
if state == "AWAKE":
    spontaneously:
        send("wake-up!") to N()
    receiving(msg):
        None

if state == "ASLEEP":
    receiving(msg):
        send("wake-up!") to N() - sender
        state = "AWAKE"
```

Si osserva che il protocollo WFLOOD è molto simile al protocollo FLOODING studiato nella lezione precedente. In particolare si osserva che il problema del broadcast è un caso

particolare del problema del wake-up, nel quale è presente, nella configurazione iniziale, un solo nodo sveglio. In altri termini, il problema del wake-up è come il problema broadcast in cui però possono esserci più nodi che posseggono l'informazione da condividere con gli altri nodi.

Correttezza

La correttezza del WFLOOD segue direttamente dalla correttezza del FLOODING per il broadcast. Si dà una dimostrazione esplicita, analoga a quella per il flooding.

Sia $\{L_0, L_1, \dots, L_h\}$ una partizione a livelli di V , dove

$$L_0 = W$$

$$L_d = \{u \in V - \bigcup_{0 \leq i \leq d-1} L_i : \exists v \in L_{d-1} \text{ t.c. } (v, u) \in E\}$$

per $d = 1, \dots, h$. Dalla definizione di L_d si osserva esplicitamente che

$$\bigcup_{i=0}^h L_i = V \wedge L_i \cap L_j = \emptyset \forall i \neq j$$

L_d è quindi l'insieme contenente tutti i nodi y tali per cui

$$\exists x \in L_0 : d(x, y) = d \wedge x = \operatorname{argmin}_{s \in W} d(s, y).$$

Si dimostra per induzione su d che esiste un tempo t_d tale che tutti i nodi in L_d hanno ricevuto almeno una volta un messaggio di wake-up, ovvero che lo stato dei nodi appartenenti ad L_d al tempo t_d sia *awake*.

Per $d = 0$ è banale in quanto $L_0 = W$. Si considera come caso base $d = 1$. Per definizione del protocollo, in un certo istante finito gli agenti in W inviano in maniera spontanea il messaggio a tutti i nodi in $N = \bigcup_{v \in W} N(v)$ e per definizione di L_d , vale $L_1 \equiv N$. Per l'assunzione dell'affidabilità totale, non ci saranno fallimenti durante la trasmissione, quindi per l'assioma di finite transmission delay, per $d = 1$ esiste un istante $t_1 \in \mathbb{R}^+$ entro il quale tutti i nodi in L_1 risulteranno nello stato *awake*.

Sia l'ipotesi vera fino ad un certo $d - 1$: si dimostra che l'ipotesi è vera anche per d . Formalmente, si mostra quindi che

$$\forall x \in L_d, \exists t_d \in \mathbb{R}^+ : \text{status}(x) = \text{awake}$$

Si consideri un qualsiasi nodo $x \in L_d$ che non si trovi già nello stato *awake*. Per definizione della partizione, esiste un nodo $y \in L_{d-1}$ tale che $(y, x) \in E$. Se $y \in W$, allora $x \in L_1$ e l'ipotesi vale per il passo base. Se $y \notin W$, allora certamente y avrà ricevuto il messaggio mentre era nello stato *asleep* per l'ipotesi induttiva. Per definizione del protocollo y avrà inviato il messaggio al suo vicinato $N(y)$, di cui x fa parte. Per l'affidabilità totale, il messaggio arriva in tempo finito e non corrotto al nodo x . Infine per l'ipotesi di connettività di G , si ha che

$$\forall v \in V, \exists k \in \mathbb{N}, k \leq \text{diam}(G) : v \in L_k$$

Ciò implica che ogni nodo della rete termina nello stato `awake`.

Complessità

Si analizza ora la message e la ideal time complexity. Sia k il numero di nodi inizialmente nello stato `AWAKE`, cioè $|W| = k$.

Message Complexity

Ogni nodo in W invia un messaggio a tutti i suoi vicini, mentre ogni nodo in $V \setminus W$ invia un messaggio a tutti i suoi vicini eccetto il vicino che lo ha fatto svegliare. Dunque il numero di messaggi totali inviati è pari a

$$\begin{aligned} M(\text{WFLOOD}) &= \sum_{u \in W} |N(u)| + \sum_{u \in V \setminus W} (|N(u)| - 1) \\ &= \sum_{u \in W} |N(u)| + \sum_{u \in V \setminus W} |N(u)| - \sum_{u \in V \setminus W} 1 \\ &= \sum_{u \in V} |N(u)| - \sum_{u \in V \setminus W} 1 \\ &= 2m - (n - k) = 2m - n + k \end{aligned}$$

dalla formula si evince che il caso peggiore è quando $W \equiv V$, ovvero $k = n$: dato che ogni nodo non conosce lo stato degli altri agenti, ogni nodo manda il messaggio a tutti i propri vicini, dunque la message complexity è proprio $2m$. Viceversa, il caso migliore è quando $k = 1$, dove il protocollo WFLOOD si riduce all'esecuzione del protocollo FLOOD a singola sorgente, con message complexity $2m - n + 1$.

Per le osservazioni appena fatte, si identifica la seguente relazione

$$M(\text{Wake-up}/R) \geq M(\text{Broadcast}/R_{\text{broadcast}}) = \Omega(m)$$

Ideal time complexity

Per poter effettuare un'analisi dell'ideal time complexity, si impongono le restrizioni di clock sincronizzati e unitary bounded delay.

Si osserva che il problema broadcast è un caso particolare del problema wake-up, nel quale un solo agente appartiene a W . Tale caso identifica il worst-case in termini di complessità temporale per il problema wake-up, infatti:

$$T(\text{Broadcast}/R_{\text{broadcast}}) \geq T(\text{Wake-up}/R)$$

dove, se $|W| = 1$, allora

$$T(\text{Broadcast}/R_{\text{broadcast}}) = T(\text{Wake-up}/R)$$

essendo che i due casi coincidono.

WFLOOD in un albero

Si supponga di eseguire il protocollo WFLOOD su una rete ad albero T . Essendo T un albero, vale che $m = n - 1$. Sostituendo alla formula calcolata in precedenza, si ottiene che la message complexity di WFLOOD su T è pari a

$$M(\text{WFLOOD}/\text{Tree}) = 2m - n + k = 2(n - 1) - n + k = n + k - 2$$

In particolar modo, si osserva come la message complexity di WFLOOD su un albero dipenda anche dalla dimensione dell'insieme dei nodi inizialmente svegli:

Il caso peggiore è per $|W| = k = n$:

$$M(\text{WFLOOD}/\text{Tree}) = n + n - 2 = 2(n - 1)$$

mentre il caso migliore lo si ottiene per $k = 1$, dove la message complexity per wake up su un albero coincide con la message complexity per broadcast su un albero:

$$M(\text{WFLOOD}/\text{Tree}) = n + 1 - 2 = n - 1$$

La crescita della message complexity è quindi lineare nella dimensione dell'insieme degli initiator: aumenta al crescere di $|W|$.

Lower bound per cliques

L'utilizzo del protocollo WFlood su un grafo completo richiede uno scambio di messaggi di ordine $\mathcal{O}(n^2)$:

$$M(\text{WFLOOD}/\text{Clique}) = 2m - n + k = 2\frac{n(n-1)}{2} - n + k = n(n-1) - n + k = \mathcal{O}(n^2)$$

In particolar modo, si può dimostrare che

$$M(\text{Wake-up}/\text{Clique}) = \Omega(n^2)$$

Ciò implica che WFLOOD è un protocollo ottimale in termini di message complexity per il problema wake-up, e, in particolar modo, si ha che:

La message complexity di wake-up sotto R in un grafo completo è $\Theta(n^2)$.

Per ridurre il numero di messaggi, è necessario imporre un'ulteriore restrizione oltre ad R e K (dove $K \equiv$ Il grafo sottostante alla rete è completo):

$$ID \equiv \text{Initial Distinct values}$$

ossia, la rete presenta un unique labeling (in altri termini, ogni nodo della rete è etichettato univocamente.) Si vuole dimostrare il seguente lower bound:

Sotto le restrizioni $R \cup K \cup ID$ vale che

$$M(\text{Wake-up}) \geq \frac{n \log n}{2} = \Omega(n \log n)$$

Per fare ciò, si costruisce un caso "sfavorevole" in termini di scambio di messaggi, ma possibile, che un qualsiasi protocollo che risolve il problema può incontrare, e si mostra che, in tale caso, verranno scambiati un numero di messaggi di ordine $\mathcal{O}(n \log n)$. Tale lower

bound vale anche se è presente un message ordering. Per semplicità, si assume che n sia una potenza di 2; il risultato vale anche senza quest'ultima assunzione.

Per costruire un caso "sfavorevole" per un protocollo arbitrario A che risolve il problema, si fa riferimento alla tecnica dell'adversarial strategy:

si assume che un avversario possa influenzare lo scenario esecutivo del protocollo, affinché gli agenti, i quali rispettano le procedure definite in tale protocollo, utilizzino il maggior numero di messaggi possibile. L'avversario può effettuare le seguenti imposizioni:

1. Decide i valori iniziali delle entità (tra i quali le etichette, che però per definizione delle restrizioni devono essere distinte)
2. Decide quale entità inizia spontaneamente l'esecuzione di A , e quando.
3. Decide quando un messaggio trasmesso arriva a destinazione (ovviamente in tempo finito)
4. Decide la corrispondenza tra links ed etichette: siano $e_1, e_2, \dots, e_k$ gli archi incidenti ad x , e siano $l_1, l_2, \dots, l_k$ i corrispondenti numeri di porta che x usa per tali archi. Quando x invia un messaggio al numero di porta l , ed l non è stata ancora assegnata come etichetta a nessun link, l'avversario deciderà a quale tra i link non ancora utilizzati (ossia tali per cui nessun messaggio è stato inviato o ricevuto tramite essi) verrà assegnata l'etichetta l .

Note

Quando si invia un messaggio da un agente a più di una porta, tale azione corrisponde all'invio del messaggio su ciascuna porta, una alla volta, in ordine arbitrario.

Qualsiasi azione decisa dall'avversario può verificarsi in un'esecuzione reale del protocollo.

Due insiemi di entità $X \subseteq V$ e $Y \subseteq V$ tali che $X \cap Y = \emptyset$ si dicono connessi al tempo t se almeno un messaggio è stato scambiato tra un'entità di X ed un'entità di Y .

Si mostra quindi come un avversario può creare un "bad-case" per il protocollo A :

1. Inizialmente, l'avversario sveglierà una singola entità s , alla quale ci si riferirà come *seed*, che inizierà l'esecuzione del protocollo. Quando s decide di inviare un messaggio al numero di porta l , l'avversario sveglierà un'altra entità y ed assegnerà l'etichetta l all'arco da s ad y , ossia, pone $\lambda_s(s, y) = l$. Ritarderà poi la trasmissione del messaggio sul link sino al momento in cui y decide di inviare un messaggio su un numero di porta l' . L'avversario assegnerà poi l'etichetta l' sul link da y ad s , ossia $\lambda_y(y, s) = l'$ e fa sì che i due messaggi inviati giungano alle proprie destinazioni simultaneamente. Così facendo, ogni messaggio raggiungerà un nodo sveglio, e le due entità risulteranno connesse. Da questo punto in poi, l'avversario agirà in maniera simile: si assicurerà

sempre che i messaggi vengano inviati a nodi già svegli, e che l'insieme di nodi svegli risultino essere connessi.

2. Si consideri un'entità x che esegue l'invio di un messaggio ad una label a ancora non assegnata:

- Se x ha un arco incidente non ancora utilizzato che lo collega ad un nodo sveglio, l'avversario assegnerà a a tale link. In altre parole, l'avversario cercherà sempre di fare in modo che le entità sveglie inviino messaggi ad altre entità sveglie.
- Se tutti i link tra x e i nodi svegli sono stati utilizzati, allora l'avversario creerà un altro insieme di nodi svegli, e conetterà i due insiemi:
 1. Siano $x_0, x_1, \dots, x_{k-1} \in X$ i nodi già svegli, ordinati secondo il loro tempo di wake-up (quindi $x_0 = s$ è il seed, e $x_1 = y$ è il nodo risvegliato durante il primo scambio di messaggi). L'avversario eseguirà la seguente funzione: sceglie k nodi inattivi $z_0, z_1, \dots, z_{k-1} \in Z$ e stabilisce una corrispondenza logica tra x_j e z_j : assegna i valori iniziali alle nuove entità $z_0, z_1, \dots, z_{k-1}$, ancora inattive, in maniera tale che l'ordinamento tra tali entità sia lo stesso delle corrispondenti entità della sequenza $x_0, x_1, \dots, x_{k-1}$. Sveglia poi le nuove entità, forzandole ad avere la stessa esecuzione (ossia stesso scheduling e stessi ritardi) delle corrispondenti entità precedentemente sveglie (quindi, z_0 verrà svegliato per primo, il suo primo messaggio verrà inviato a z_1 , il quale a sua volta verrà svegliato prima dell'arrivo di tale messaggio ed invierà un messaggio a z_0 , e i ritardi dei messaggi coinvolti in questa operazione verranno scelti dall'avversario in maniera tale che questi raggiungano la propria destinazione nello stesso istante, e così via...).
 2. L'avversario assegnerà poi la label a al link che collega x alla sua corrispondente entità z nel nuovo insieme Z ; Il messaggio rimarrà in transito in tale link sino a quando z dovrà trasmettere un messaggio su un link non ancora utilizzato, sia questo etichettato con b , ma tutti gli archi che lo collegano con il suo insieme di entità già sveglie Z risultano essere stati già utilizzati.
 3. Quando accade ciò, l'avversario assegnerà la label b al link da z ad x , e farà in modo che i due messaggi tra x e z arrivino e vengano processati.

Riassumendo, in questa strategia, l'avversario cerca di forzare il protocollo ad inviare messaggi solamente ad entità già sveglie, e di svegliare nuove entità solo quando non può fare altrimenti. Le nuove entità appena svegliate sono uguali in numero alle entità precedentemente sveglie, le quali vengono forzate dall'avversario ad eseguire lo stesso scambio di messaggi avvenuto tra le entità precedentemente sveglie prima che i due insiemi possano iniziare a comunicare. Quando ciò accade, si dice che l'avversario ha iniziato una nuova fase.

Si analizza ora la situazione creata dall'avversario con questa strategia, e si analizza il costo del protocollo nelle corrispondenti esecuzioni. Sia $Active(i)$ l'insieme delle entità sveglie nella fase i , e sia $New(i) = Active(i) - Active(i-1)$ l'insieme di entità che l'avversario ha svegliato in tale fase. Inizialmente, in $Active(0)$ si trova solamente il seed s . Le entità

svegliate nella fase i sono uguali in numero alle entità precedentemente sveglie, ossia

$$|New(i)| = |Active(i-1)|.$$

Sia $\mu(i-1)$ il numero totale di messaggi scambiati prima dell'attivazione delle nuove entità nella fase i . L'avversario forza le nuove entità ad avere la stessa esecuzione compiuta dalle entità in $Active(i-1)$, portando così allo scambio di $\mu(i-1)$ messaggi prima che i due insiemi vengano connessi. Quindi, il numero totale di messaggi scambiati prima che avvenga la comunicazione tra i due insiemi è $2\mu(i-1)$.

Si vuole determinare ora, una volta che è iniziata la comunicazione tra i due insiemi, il numero di messaggi che vengono trasmessi prima dell'inizio della prossima fase.

La risposta esatta dipende dal protocollo A , ma, indipendentemente dal protocollo utilizzato, l'avversario non inizierà la nuova fase $i+1$ a meno che non venga forzato, e ciò avviene solo se un'entità x esegue un invio su un'etichetta l , dove l non è stata ancora assegnata, e tutti i link che collegano x alle altre entità già nello stato di awake sono stati già utilizzati.

Questo vuol dire che x deve aver inviato o ricevuto un messaggio da tutte le entità in $Active(i) = Active(i-1) \cup New(i)$. Si distinguono due casi:

- Se $x \in Active(i-1)$, allora, tra tutti i messaggi, quelli tra x e le entità in $New(i)$ sono stati scambiati solamente nella fase i , essendo che queste entità non erano attive in precedenza. Questo vuol dire che almeno $|New(i)| = |Active(i-1)|$ ulteriori messaggi vengono inviati prima della fase $i+1$.
- Se $x \in New(i)$, allora i messaggi inviati da x sono stati trasmessi tutti nella fase i , essendo che x non era sveglio in precedenza, e quindi anche i messaggi diretti verso $Active(i-1)$; in altre parole, anche in questo caso $|New(i)| = |Active(i-1)|$ ulteriori messaggi vengono inviati prima della fase $i+1$.

In sintesi, il costo totale $\mu(i-1)$ prima della fase i è quindi raddoppiato, ed almeno $|Active(i)|$ ulteriori messaggi vengono inviati prima dell'inizio della fase $i+1$. Quindi

$$\mu(i) \geq 2\mu(i-1) + |Active(i-1)|$$

Essendo che le entità svegliate raddoppiano in ogni fase, ed inizialmente solo il seed è attivo, allora $|Active(i)| = 2^i$. Quindi, osservando che $\mu(0) = 0$, si ha che

$$\mu(i) \geq 2\mu(i-1) + 2^{i-1} \geq i 2^{i-1}$$

Il numero totale di fasi è esattamente $\log n$, essendo che il processo di risveglio raddoppia il numero di nodi awake in ciascuna fase. Quindi, con questa strategia, un avversario può forzare un qualsiasi protocollo a trasmettere $\mu(\log n)$ messaggi. Essendo

$$\mu(\log n) \geq \frac{n \log n}{2}$$

segue che ogni protocollo per il problema Wake-up trasmetterà $\Omega(n \log n)$ messaggi nel caso peggiore, anche se le entità hanno identificatori distinti.